VIVA QUESTION BANK

Viva Preparation

LPU Placement Intelligence Platform

Every figure below is taken from the project's own artifacts: `reports/model-evaluation/`, `reports/teacher-brief-xgboost/`, `backend/placement_model.joblib`, `reports/data-audit/` and `docs/TEACHER_REQUIREMENTS_AUDIT.md`. Nothing here is estimated.

Six numbers to memorise cold

Number	Meaning
96.08% (49/51)	Deployed Gradient Boosting, single stratified hold-out
95.9% ± 2.2	AdaBoost, best in repeated 3x5 cross-validation
76.9% / 51.4%	Rule-based baseline / majority-class baseline
0.995	Best ROC-AUC (AdaBoost, cross-validation)
77.9%	CGPA share of Gini importance
251	Rows after removing 2 duplicates (from 253)

A. The accuracy question (they will start here)

1. Your model shows 96% accuracy. Explain exactly what that number is.

It is the accuracy of the deployed Gradient Boosting classifier on a stratified 20% hold-out of 51 students it never saw during training — 49 correct, 2 errors (1 false positive, 1 false negative). It means: given nine self-reported competency inputs, the model reproduces the `placement_ready` label 96 times out of 100.

2. Is 96% your cross-validation result or one lucky split?

Both are reported, because one split hides variance. Repeated stratified 3x5 cross-validation over all 251 rows gives AdaBoost 95.9% ± 2.2, Gradient Boosting 94.7% ± 3.0, Random Forest 93.9%, Logistic Regression 94.0%. So the honest statement is "94-96%, with roughly ±3 points of split variability" — not "exactly 96%".

3. Why is accuracy this high? Isn't the problem too easy?

Partly, yes, and I will say so. The label is rule-consistent: CGPA alone carries 77.9% of the importance and active backlogs 7.3%, so the classes separate almost linearly. That is exactly why we added a 76.9% threshold baseline first — the model's real contribution is the extra ~19 points over a hand-written rule, plus calibrated probabilities and per-student explanations.

4. If accuracy is high, why compare eight algorithms?

To prove the choice was not arbitrary. Same data, same split, same seed 42, five metrics each: Gradient Boosting 96.08%, Random Forest 94.12%, Decision Tree 94.12%, Naive Bayes 92.16%, AdaBoost 92.16%, Logistic Regression 92.16%, KNN 92.16%, SVM 90.20%. A single model would tell the examiner nothing about whether the approach was appropriate.

5. Accuracy alone is not sufficient — prove you know that.

Correct, and that is why we report balanced accuracy, precision, recall, F1, ROC-AUC, a confusion matrix, error analysis, and probability calibration. Calibration is the clearest evidence: raw Gradient Boosting had a log loss of 0.816; after sigmoid calibration on out-of-fold predictions it fell to 0.200, with Brier 0.053 to 0.046, at essentially unchanged AUC (0.985 to 0.983). Accuracy did not change; the probability quality did.

6. What is the model predicting — placement or readiness?

Readiness. The target is `placement_ready`, a prototype label whose construction is unverified. We deliberately do not claim "this student will be placed" and we do not call the output a placement probability in production copy. It is a readiness ranking signal plus a skill-gap report.

7. AdaBoost has the best CV score. Why is Gradient Boosting the deployed model?

Because the deployed artifact was selected on the frozen hold-out protocol used for the 8-model benchmark, where Gradient Boosting won at 96.08% with the fewest errors (2), and it was then integrated, versioned and calibrated. The cross-validation study came later and showed AdaBoost ahead by 1.2 points — within the ±2-3 point split noise, and AdaBoost's higher variance on unseen data is a known weakness. We state both numbers rather than hiding the disagreement. If we retrain for production, AdaBoost is the candidate to re-test.

B. Data

8. Where did the data come from?

Seven CSVs, 4,048 rows total, documented in `dataset/README.md` with source attribution. The live nine-feature model trains on `students.csv`: 253 rows, nine competency columns, target `placement_ready`. The other six are external benchmark and secondary-analysis files, some Kaggle-derived.

9. 253 rows is tiny. Can you defend that?

For nine numeric features it is workable — roughly 28 rows per feature, and repeated 3x5 cross-validation gives 15 estimates per model so we are not reading a single number. But it is not enough to claim generalization, and the CV spread of ±3 points reflects split variability only, not real-world performance. We list this as a limitation instead of overselling it.

10. You have seven datasets but train on one. Isn't that wasteful?

No — they have different populations and different target definitions, so merging them would be invalid without a documented schema mapping, as recorded in the dataset audit and requirements audit. We used them for external benchmarking instead: on `college_placement_engineering.csv` (2,966 rows, reduced to 1,137 after removing 1,829 exact duplicates), Random Forest reached 79.2% accuracy and 0.852 ROC-AUC versus Logistic Regression 72.8% and 0.832. That is our evidence that the pipeline generalizes to a different, harder population — and that the 96% figure is dataset-specific.

11. What did EDA show?

Near-balanced classes (129 vs 122 after dedupe, so a 51.4% majority), no missing values in the primary set, a strong monotone CGPA-readiness relationship, hard threshold effects around 7.0-7.5 CGPA, and backlogs acting as a binary disqualifier. The class balance is why F1 and balanced accuracy are meaningful here, and why we did not need SMOTE.

12. Missing values and outliers — how were they handled?

`data_pipeline.py` validates schema, label domain and missingness before training, and fails loudly rather than silently imputing. In the primary set there is nothing to impute. Outliers were inspected, not clipped: CGPA is bounded 0-10 and scores 0-100 by construction, and tree ensembles are insensitive to monotone outliers, so clipping would have destroyed real signal at the eligibility boundary.

13. Why exactly 2 rows removed?

They were exact duplicates. Duplicates inflate apparent accuracy because the same row can land in both train and test, so removal is explicit and logged (253 to 251) rather than done quietly inside a split call. The audit report flags 1,829 duplicates in the engineering dataset as a blocking finding — the same principle applied consistently.

14. Is there label leakage?

We checked for it explicitly. The label is not a column-derived formula we feed back in, but it is rule-consistent with CGPA and backlogs, which is a circularity risk rather than classic leakage: a model can score highly by re-learning the rule that generated the label. We disclose that, keep the threshold baseline visible as the reference point, and flag the label as unverified pending owner confirmation.

C. Preprocessing and features

15. Did you scale the features?

For the canonical comparison, no scaler was applied, because seven of the eight candidates are tree-based and invariant to monotone feature scaling. Where it matters: SVM and KNN are scale-sensitive, and mixing CGPA (0-10) with scores (0-100) lets the score columns dominate distance and kernel computations — a plausible reason they scored lowest at 88.0% and 90.0%. That is a stated limitation, and the fix is a `StandardScaler` inside a pipeline for those two models. The external benchmark trainer already uses `Pipeline` with `ColumnTransformer` and `StandardScaler` for exactly this reason.

16. What feature engineering did you do, and why isn't it in the live model?

In the XGBoost research workflow we created `coding_technical_mean` (mean of coding and technical scores) and `experience_count` (projects plus internships). They are not in the active model, deliberately: the deployed artifact stays on the original nine inputs so the API contract and the existing model file remain valid. Promoting engineered features would require a retrain, re-validation and an API version bump — we did not want an unevaluated model silently going live.

17. Feature selection or dimensionality reduction?

Not needed with nine low-collinearity inputs, and PCA would have destroyed the interpretability we rely on for the skill-gap panel. We did the diagnostic instead: Gini importance across the ensemble ranks CGPA 77.91%, backlogs 7.26%, certifications 4.45%, communication 2.87%, aptitude 2.84%, coding 2.44%, technical 1.18%, projects 1.06%, internships 0.00%. Internships at 0% reflects a near-constant distribution in the training cohort, not irrelevance in reality — we say that explicitly rather than dropping the feature.

18. Class imbalance handling?

Balanced enough (51.4 / 48.6) that resampling would add noise, not information. We handled it in evaluation instead — balanced accuracy and F1 alongside accuracy, `class_weight="balanced"` on Random Forest, and stratified splits everywhere so every fold keeps the class ratio.

D. Models, ensembles and tuning

19. Why tree ensembles? Why not deep learning?

Nine tabular features, 251 rows, monotone thresholds. A neural network would have far more parameters than data points, would overfit, and would cost us the per-feature attribution the product is built on. We justify the family, not just the winner: linear, instance-based, margin, probabilistic and ensemble models were all tested, and the boosted ensembles won because placement eligibility behaves like a set of interaction thresholds.

20. Bagging versus boosting — which won and why?

Boosting, narrowly. Bagging (Random Forest, 250 trees) reduces variance by averaging independent trees; boosting fits trees sequentially to the residual errors, reducing bias. With hard cutoffs where a handful of students sit right at the boundary, bias reduction was worth more than variance reduction. Concretely: Gradient Boosting 96.08% with 2 errors, Random Forest 94.12% with 3.

21. How did you tune hyperparameters?

Grid search with cross-validation on the training split only: `n_estimators` in {80, 160}, `max_depth` in {2, 3}, `learning_rate` in {0.05, 0.1}, 3-fold stratified, selected on balanced accuracy because accuracy alone would let the majority class drive the choice. Best: depth 2, learning rate 0.05, 160 trees, cross-validated balanced accuracy 0.9399. Depth 2 winning is itself informative — the boundary is simple, and deeper trees just memorized noise.

22. Did tuning leak the test set?

No. Selection used only the 200-row training portion; the 51 held-out rows were touched once, at the end. Final tuned XGBoost test result: 88.2% (45/51), F1 0.88, ROC-AUC 0.977, against 51.0% for the majority baseline — and we report that lower number even though the untuned Gradient Boosting scored higher, because the tuned model is the one that followed the honest protocol.

23. What were your baselines?

Two, not one: majority class at 51.4%, and a domain rule applying the institutional thresholds (CGPA 7.5 or above, coding 75 or above, zero active backlogs, and so on) at 76.9% accuracy. The rule baseline is the meaningful one — it is what a placement officer does by hand, and it has 0.526 recall, missing nearly half of ready students. Beating that is the actual result.

E. Explainable AI

24. Your model is 78% CGPA. Isn't that a red flag?

It is a finding, and we treat it as one. It says the prototype label is dominated by academic eligibility, which is honest about the data and also a warning: a fairness review would need to check whether CGPA is acting as a proxy for something else, and subgroup performance is unverified. The product does not rely on importance alone — the skill-gap matrix uses fixed industry thresholds so the advice is auditable.

25. Which XAI method did you use — SHAP or LIME?

Both feature importance (Gini, global) and SHAP (TreeExplainer, local, on demand through `POST /api/explain/shap`). LIME is not implemented; the brief allows either. SHAP values are labelled as raw log-odds in the UI, not probabilities, because mixing the two is a common misreading. SHAP is unavailable for models the explainer cannot wrap, and the UI says so rather than showing a blank chart.

26. Give me a local explanation for one student.

For a student just below the line, the panel shows the one-feature counterfactual: raising CGPA from 6.8 to 7.6 moves predicted readiness from about 0.42 to about 0.78, while adding a project moves it far less. That is what makes the output actionable — it tells the student which lever actually matters, not just a score.

F. Pipeline and deployment

27. Show me the pipeline. Is it reproducible?

Five stages, all scripted: validate (`data_pipeline.py`), evaluate (`compare_models.py` into `reports/model-evaluation/`), train and persist (`train_model.py` into `placement_model.joblib`), version and register (`model_registry.py`), then optional offline tuning (`tune_xgboost.py` into `reports/teacher-brief-xgboost/`). Reproducibility is anchored on the dataset SHA-256 `e9fe3511...`, a fixed seed of 42, and pinned requirements. Nothing is promoted automatically from the research experiment.

28. Is it actually deployed, or does it only run locally?

It runs as a real service, not a notebook: a React + Vite frontend and a Flask REST API (`/api/predict`, `/api/model-info`, `/api/explain/shap`, `/api/cohort-analytics`), served in containers. `docker compose up --build` starts the development stack on ports 5173 and 5000; `docker-compose.production.yml` runs Gunicorn with 2 workers x 4 threads, non-root containers, unprivileged Nginx with SPA fallback and `/api/` reverse proxy, and readiness-gated startup. What is not done is public hosting with TLS, institutional identity and a managed database — I will say that plainly rather than overclaim.

29. What happens on a version mismatch between model file and library?

A real issue: the artifact was serialized with scikit-learn 1.9.1 and the review environment had 1.9.0, which warns on load. The mitigation is to pin the training environment in the Docker image and retrain inside the deployment image, which is why `requirements.txt` and the Dockerfile matter as much as the model file.

30. What are the limits of this deployment?

Identity and tenancy are prototype-grade: a SQLite history volume, no OIDC, no role-based authorization beyond the application's own checks, no backups or monitoring. Input validation is enforced at the API boundary (CGPA 0-10, scores 0-100, non-negative counts) so malformed requests cannot silently produce garbage predictions. Scaling beyond a department needs a managed database, TLS, and an approved data-retention policy.

G. Trap questions

> **31. I give you 5,000 verified LPU records tomorrow. What breaks?**
> The feature schema holds, so retraining is straightforward — but everything downstream must be re-derived: class balance may shift, the 0.9399 tuned CV score is meaningless on new data, thresholds may need re-fitting, calibration must be redone, and subgroup fairness becomes testable for the first time. It would be a new evaluation, not a rerun.

> **32. What would you do differently with more time?**
> Three things: fix the label's provenance so accuracy means placement outcomes instead of a rule, add `StandardScaler` inside a pipeline for the scale-sensitive models, and complete the Supabase migration with real authentication so it is deployable beyond a demo.

> **33. What did each of you actually build?**
> Answer with the split on slide 18 — data and ML (dataset audit, pipeline, model comparison, tuning, calibration), backend and deployment (Flask API, authentication, Docker, Nginx, Gunicorn), frontend and XAI (React dashboards, skill-gap matrix, SHAP panel, telemetry). Each person should be able to name one artifact they own and one number they produced.

Delivery advice

> Open with the 76.9% rule baseline and the 51.4% majority baseline before saying 96%, and always say "94-96% across repeated cross-validation" rather than a bare 96%. Volunteering that the label is a prototype and the 96% is same-dataset makes every subsequent question easier — it is the difference between a team that trained a model and a team that understands one.